

DecidArch — Shinobi Arena

Resumen de reglas. El reglamento completo está en [DecidArch-GameRules.pdf](#)

Contenido del mazo

- 1 Project Card · 3 Stakeholder Cards · 10 Concern Cards · 6 Event Cards
- 1 Decision Preparation Template por jugador · 1 Decision Taking Template por grupo · 1 Scoring Sheet
- Equipo de 2 a 4 jugadores. Duración: 30 minutos. Hace falta un lápiz y un reloj.

Preparación

- Dejar a la vista la Project Card y las Stakeholder Cards.
- Formar dos mazos boca abajo y barajados: uno de Concern Cards y otro de Event Cards.
- Entregar una Decision Preparation Template a cada jugador y una Decision Taking Template al grupo.
- Todas las QA-Priority arrancan en 0. Son las Event Cards las que las modifican.

Turno

- El jugador de turno roba una Concern Card.
- Cada jugador, de forma independiente, elige una opción y la anota con su rationale en su plantilla individual.
- El grupo discute y decide una sola opción, y la registra en la Decision Taking Template.
- Pasa el turno al siguiente jugador.

Event Cards

- Al terminar la ronda, cuando todos han jugado su turno, se roba una Event Card.
- El evento cambia el contexto del proyecto. Sus efectos duran el resto de la partida.
- Hay que reconsiderar las decisiones ya tomadas a la luz del cambio.
- Si no quedan Event Cards, se sigue jugando sin robar.

Opciones propias y símbolo <?>

- Los jugadores pueden proponer opciones nuevas en cualquier momento, anotándolas en la columna de opción.
- También pueden corregir los impactos de una opción existente, justificándolo en la columna Rationale.
- La lista de impactos de cada opción termina en "..." porque no es cerrada.

Fin de la partida. Se juega hasta agotar los 30 minutos o resolver todas las Concern Cards. Después se calcula el resultado con el Scoring Sheet: si algún valor de C queda bajo cero, el equipo pierde, sin importar el puntaje acumulado.

Project Card

Project:
Shinobi Arena

Purpose:

Shinobi Arena es un videojuego de peleas 2D de uno contra uno, con vista lateral y control por teclado, que se instala y se ejecuta localmente. El jugador compite en el Torneo de los Cinco Reinos enfrentando rivales sucesivos, cada uno con un estilo de combate distinto, hasta el oponente final. Se construye en Greenfoot con Java y debe portarse a Unity más adelante.

Stakeholder Card

Project:
Shinobi Arena

Stakeholder:
Jugador

Goal:

El Jugador quiere sentarse frente al teclado y tener un combate que se sienta justo: que el personaje responda a lo que ordeno, entender por qué perdió cuando pierde, y poder retomar el juego semanas después sin reaprender los controles.

Quality Attributes:

Satisfaction — Fluidez y respuesta del combate: que el personaje haga lo ordenado, cuando se ordena. **(QA-Priority: 0)**

Reliability — Ausencia de estados rotos, personajes trabados y comportamiento impredecible. **(QA-Priority: 0)**

Learnability — Cuánto tarda un jugador nuevo en entender los controles y las mecánicas básicas. **(QA-Priority: 0)**

Accessibility — Cuánto recuerda el jugador de las mecánicas al volver después de no jugar. **(QA-Priority: 0)**

Performance — Coste de cálculo por frame y estabilidad de la velocidad en pantalla. **(QA-Priority: 0)**

Stakeholder Card

Project:
Shinobi Arena

Stakeholder:
Equipo de desarrollo

Goal:

El Equipo construye el prototipo, lo mantiene durante el semestre y debe portarlo a Unity más adelante. Necesita poder depurar rápido y agregar contenido sin reescribir lo que ya funciona.

Quality Attributes:

Portability — Esfuerzo para llevar el juego de Greenfoot a Unity y a otros sistemas operativos. **(QA-Priority: 0)**

Modifiability — Esfuerzo para agregar un personaje, un ataque o una arena sin tocar lo existente. **(QA-Priority: 0)**

Testability — Poder comprobar una regla de combate sin tener que jugar una partida completa. **(QA-Priority: 0)**

Analysability — Facilidad para leer el código y localizar la responsabilidad de cada clase. **(QA-Priority: 0)**

Reliability — Ausencia de estados rotos, personajes trabados y comportamiento impredecible. **(QA-Priority: 0)**

Stakeholder Card

Project:
Shinobi Arena

Stakeholder:
Revisor técnico

Goal:

El Revisor abre el proyecto sin conocimiento previo y debe poder explicar qué hace cada clase, verificar que el comportamiento común no esté duplicado y comprobar las reglas del combate de forma aislada.

Quality Attributes:

Analysability — Facilidad para leer el código y localizar la responsabilidad de cada clase. **(QA-Priority: 0)**

Modifiability — Esfuerzo para agregar un personaje, un ataque o una arena sin tocar lo existente. **(QA-Priority: 0)**

Testability — Poder comprobar una regla de combate sin tener que jugar una partida completa. **(QA-Priority: 0)**

Reliability — Ausencia de estados rotos, personajes trabados y comportamiento impredecible. **(QA-Priority: 0)**

Concern Card

Concern-ID: **1**

prioridad 5,00 · C-02

Concern:

El jugador y el enemigo comparten vida, chakra, gravedad y daño, y solo se diferencian en quien decide sus acciones. Cómo debe repartirse ese comportamiento entre clases?

Options:

- 1 Dos clases independientes: Player y Enemy heredan de Actor por separado, cada una con su propia copia de la lógica de vida, chakra y física.
Performance (+), Modifiability (-), Analysability (-), Testability (-), ...
- 2 Clase abstracta Fighter: concentra vida, chakra, física y daño; Player y Enemy la extienden y solo redefinen como se decide la acción.
Modifiability (+), Analysability (+), Reliability (+), Testability (-), ...
- 3 Composición por componentes: Fighter contiene objetos Movimiento, Combate y Salud intercambiables entre si.
Modifiability (+), Testability (+), Portability (+), Analysability (-), Performance (-), ...

...

Concern Card

Concern-ID: **2**

prioridad 4,80 · C-01

Concern:

Un luchador puede estar quieto, caminando, saltando, atacando, bloqueando, golpeado o derrotado, y hay combinaciones prohibidas: quien esta siendo golpeado no puede atacar. Cómo se representa su estado?

Options:

- 1 Banderas booleanas dentro de Fighter: atacando, saltando y bloqueando consultadas con if anidados.
Performance (+), Reliability (-), Modifiability (-), Analysability (-), Satisfaction (-), ...
- 2 Un enum FighterState con un switch en act() que decide que acciones se permiten en cada estado.
Reliability (+), Satisfaction (+), Analysability (+), Performance (+), Modifiability (-), ...
- 3 Patron State: una clase por estado (IdleState, AttackingState, HurtState...), cada una decide sus propias transiciones.
Reliability (+), Modifiability (+), Analysability (+), Satisfaction (+), Testability (+), Performance (-), ...

...

Concern Card

Concern-ID: **3**

prioridad 4,30 · C-04

Concern:

Un golpe conecta durante pocos cuadros y solo en una zona por delante del personaje; un proyectil impacta en otro momento y con otra forma. Cómo se detecta que un ataque impacto?

Options:

- 1 Intersección de los sprites completos con getIntersectingObjects mientras dura el ataque.
Performance (+), Reliability (+), Satisfaction (-), Modifiability (-), ...
- 2 Un actor Hitbox temporal que el ataque crea durante sus cuadros activos y destruye al terminar; Projectile reutiliza el mismo mecanismo.
Satisfaction (+), Modifiability (+), Learnability (+), Analysability (+), Performance (-), ...
- 3 Cálculo geométrico de rectangulos dentro del árbitro del combate, sin crear actores adicionales.
Performance (+), Testability (+), Portability (+), Satisfaction (+), Analysability (-), ...

...

Concern Card

Concern-ID: **4**

prioridad 4,20 · C-05

Concern:

Alguien debe aplicar el daño, restar chakra, decidir si el bloqueo lo reduce, detectar que la vida llego a cero y declarar el resultado. Que componente arbitra las reglas del combate?

Options:

- 1 FightWorld hace todo: crea los actores, aplica el daño y decide el final.
Performance (+), Analysability (-), Modifiability (-), Testability (-), Portability (-), ...
- 2 Un CombatManager colaborador: FightWorld le delega las reglas; el manager no sabe dibujar ni leer teclado, y no depende de la API de Greenfoot.
Analysability (+), Modifiability (+), Testability (+), Portability (+), Reliability (+), Performance (-), ...
- 3 Un CombatManager Singleton accesible desde cualquier clase.
Performance (+), Testability (-), Reliability (-), Modifiability (-), Portability (-), ...

...

Concern Card

Concern-ID: **5**

prioridad 3,70 · C-09

Concern:

El juego tiene pantalla de inicio, combate, victoria, derrota y reinicio. Cómo se organiza el paso entre esas pantallas?

Options:

- 1 Un solo World que cambia su fondo y dibuja el menu o el combate segun una variable de estado interna.

Performance (+), Analysability (- -), Modifiability (- -), Reliability (-), Learnability (-), ...

- 2 Tres World separados (MenuWorld, FightWorld, EndWorld) enlazados con Greenfoot.setWorld().

Analysability (+ +), Modifiability (+ +), Reliability (+ +), Learnability (+ +), Portability (-), ...

- 3 Un World con un objeto GameState que delega el dibujado y el input al estado activo.

Portability (+ +), Modifiability (+), Testability (+), Reliability (+), Analysability (-), Learnability (-), ...

...

Concern Card

Concern-ID: **6**

prioridad 3,60 · C-08

Concern:

Kaien usa técnicas de fuego, Raiko es eléctrico y veloz, Sora ataca a distancia con viento. Cómo se representa la diferencia de estilo y habilidades entre personajes?

Options:

- 1 Un if por personaje dentro de Fighter.usarEspecial(), consultando el nombre del personaje.

Performance (+), Modifiability (- -), Analysability (- -), Accessibility (-), ...

- 2 Una subclase por personaje que redefine usarEspecial() y sus estadísticas base.

Modifiability (+ +), Analysability (+ +), Accessibility (+), Reliability (+), Testability (-), ...

- 3 Un objeto Tecnica intercambiable que el luchador contiene por composición y puede cambiarse en caliente.

Modifiability (+ +), Testability (+ +), Accessibility (+ +), Analysability (-), Performance (-), ...

...

Concern Card

Concern-ID: **7**

prioridad 3,60 · C-03

Concern:

El enemigo debe acercarse, retroceder, atacar a distancia adecuada, bloquear de manera ocasional y usar su especial, sin caer en una secuencia fija. Dónde vive esa lógica de decisión?

Options:

- 1 Una cadena de if sobre la distancia al jugador, escrita dentro de Enemy.act().

Performance (+), Modifiability (- -), Testability (- -), Satisfaction (-), Accessibility (-), ...

- 2 Una clase EnemyAI separada a la que Enemy delega: la IA devuelve una acción y el enemigo la ejecuta.

Modifiability (+ +), Testability (+ +), Analysability (+), Portability (+), Performance (-), ...

- 3 Un autómata de estados de IA (Perseguir, Atacar, Retroceder, Esperar) con cooldowns y transiciones probabilísticas.

Satisfaction (+ +), Accessibility (+ +), Modifiability (+), Reliability (+), Performance (-), Analysability (-), ...

...

Concern Card

Concern-ID: **8**

prioridad 2,90 · C-12

Concern:

El prototipo necesita dos arenas que se diferencian en fondo, altura del suelo, límites laterales y posiciones iniciales. Cómo se configura un escenario de combate?

Options:

- 1 Una subclase de World por arena: BosqueWorld y TemploWorld.

Reliability (+), Performance (+), Modifiability (-), Analysability (-), ...

- 2 FightWorld parametrizado con un objeto Arena que aporta fondo, suelo, límites y puntos de aparición.

Modifiability (+), Analysability (+), Testability (+), Portability (+), Reliability (-), ...

- 3 Arenas definidas en un archivo de datos que se lee al iniciar el combate.

Modifiability (+ +), Portability (+), Testability (+), Reliability (- -), Analysability (-), ...

...

Concern Card

Concern-ID: **9**

prioridad 2,80 · C-07

Concern:

El prototipo necesita un jugador y dos enemigos distintos, y más adelante posiblemente más personajes. Cómo se crean los luchadores y sus variantes?

Options:

- 1 Con new directo en FightWorld, escribiendo ahí los parámetros de cada personaje.
Performance (+), Modifiability (- -), Analysability (-), Testability (-), ...
- 2 Una FighterFactory con un método que recibe el tipo de personaje y devuelve el Fighter ya configurado.
Modifiability (+ +), Analysability (+ +), Testability (+), Performance (-), ...
- 3 Estadísticas en un archivo de configuración externo que la fabrica lee al crear cada luchador.
Modifiability (+ +), Testability (+ +), Portability (+), Reliability (-), Analysability (-), ...

...

Concern Card

Concern-ID: **10**

prioridad 2,60 · C-06

Concern:

La barra de vida y la de chakra deben reflejar el estado del luchador en todo momento. Cómo se mantienen sincronizadas con el?

Options:

- 1 Polling: FightWorld pregunta getVida() a cada luchador en cada act() y redibuja las barras.
Analysability (+), Modifiability (-), Performance (-), Satisfaction (-), ...
- 2 Observer: el Fighter notifica a sus observadores cuando su vida o chakra cambia, y las barras se actualizan solas.
Modifiability (+ +), Performance (+), Satisfaction (+), Learnability (+), Analysability (-), ...
- 3 Referencia directa: cada Fighter guarda un puntero a su HealthBar y la actualiza al recibir daño.
Performance (+ +), Satisfaction (+), Modifiability (- -), Analysability (-), ...

...

Event Card

Title:

Playtest: los golpes no se sienten

Description:

Quienes prueban el prototipo reportan que reciben daño sin ver el golpe conectar, y que a veces el personaje queda trabado a mitad de un ataque.

La Satisfaction del Jugador sube a QA-Priority 3 si no lo estaba. Reconsideren los concerns 2 y 3.

Event Card

Title:

El porteo a Unity se adelanta

Description:

La siguiente unidad del proyecto exige rehacer el juego en Unity. Todo lo que dependa de la API de Greenfoot habra que reescribirlo desde cero.

La Portability del Equipo de desarrollo sube a QA-Priority 3. Reconsideren los concerns 4 y 5.

Event Card

Title:

Se incorpora un desarrollador nuevo

Description:

Una persona sin contexto se suma al equipo y debe poder trabajar sobre el código existente sin que se lo expliquen entero.

La Analysability del Equipo de desarrollo sube a QA-Priority 3. El Revisor técnico mantiene la suya. Reconsideren el concern 1.

Event Card

Title:

Torneo de exhibición en la universidad

Description:

El juego se mostrara en una feria abierta donde la gente juega una sola partida, de pie y sin explicacion previa.

La Learnability del Jugador sube a QA-Priority 3 y la Accessibility baja a 1: importa que lo entiendan ahora, no que lo recuerden en un mes. Reconsideren los concerns 5 y 10.

Event Card

Title:

Se agregan dos personajes jugables

Description:

El alcance del proyecto crece: hay que sumar dos luchadores más con técnicas propias, reutilizando los sistemas existentes.

Si en el concern 6 eligieron la opción 1, la deuda se paga ahora: cuenten Modifiability (- -) una segunda vez. Si eligieron 2 o 3, sumen Modifiability (+). Reconsideren los concerns 6 y 9.

Event Card

Title:

Auditoría de originalidad del contenido

Description:

Se revisa que ningun nombre, técnica ni recurso grafico provenga de una obra con derechos de autor. Todo lo que no tenga origen documentado debe reemplazarse.

Los datos de cada personaje deben poder sustituirse sin tocar el resto del sistema. Reconsideren el concern 9 y registren en la columna Rationale que atributo los protegio.

Decision Preparation Template (Individual)

Cada jugador recibe una plantilla individual.

Group-ID: _____

[illegible]

Decision Taking Template (Group)

Registra las decisiones del grupo.

Group-ID: _____

[illegible]

Scoring Sheet

Se completa al terminar la partida. Fórmulas tomadas de DecidArch-ScoreSheet.pdf

Paso 1 · QA-Scores

Quality Attribute	Learnability	Satisfaction	Accessibility	Performance	Reliability	Modifiability	Analysability	Testability	Portability
A = cantidad de "+" ("+" + " cuenta dos veces)									
B = cantidad de "-" ("- -" cuenta dos veces)									
QA-Score = A - B									

Se cuentan solo los impactos de las opciones efectivamente elegidas, incluyendo los que el equipo haya agregado durante la partida.

Paso 2 · Stakeholders-Score

Stakeholder	Quality Attribute	QA-Score	QA-Priority	C = QA-Score - QA-Priority
Jugador	Satisfaction			
	Reliability			
	Learnability			
	Accessibility			
	Performance			
Equipo de desarrollo	Portability			
	Modifiability			
	Testability			
	Analysability			
	Reliability			
Revisor técnico	Analysability			
	Modifiability			
	Testability			
	Reliability			
¿Hay algún valor de C menor que 0? Si lo hay, perdieron la partida.				Si / No
Stakeholders-Score = suma de los valores de C				

Usar la QA-Priority vigente al terminar la partida, ya modificada por las Event Cards.

Paso 3 · Final Score

D = Concern Cards que quedaron sin resolver	
Final Score = Stakeholders-Score - D	

Final Score	Resultado
Menor que 0	Perdieron la partida
0 a 9	Suficiente
10 a 19	Bueno
20 a 29	Muy bueno
30 o más	Excelente